Cheepskies Technical Documentation

Table of Contents

1. Project Overview and Requirements
2. System Architecture
3. Class Diagram (UML)
4. Data Model and Database Schema
5. Program Functionality and Flow
6. Exception Handling
7. Conclusion

---

1. Project Overview and Requirements

1.1 Project Description

CheepSkies is a JavaFX-based flight booking application that connects to an Azure-hosted MySQL database. The system supports two user types: customers who search and book flights, and administrators who manage flight inventory. The application begins with a splash screen, then presents options for registration or login. Users maintain persistent reservations across sessions and can search flights using multiple criteria.

1.2 Functional Requirements

The system implements role-based access control where Admin extends Customer, providing administrators with all customer capabilities plus management privileges. Key requirements include:

Authentication & Registration: Users register with comprehensive information including name, address, email, SSN, username, password, and security question for password recovery. Login validates credentials against the database, and users can recover passwords using their security question and username.

Customer Functionality: After login, customers access a main page displaying two tables: all available flights and their personal bookings. Search functionality filters flights by departure/arrival cities, dates, and duration. Customers can book flights with validation preventing duplicate bookings, scheduling conflicts, and overbooking beyond capacity. Bookings can be cancelled, with changes persisting to the database.

Administrator Functionality: Administrators access a dedicated admin page to add, modify, and remove flights from the system. They can perform all customer operations plus manage the flight inventory through a specialized interface.

Navigation & Persistence: Users can return to the main menu from any screen. Logout properly closes sessions, and subsequent logins restore previously booked reservations from the database.

---

2. System Architecture

2.1 Architecture Pattern

The application follows a four-layer architecture: Presentation Layer (LoginController, RegistrationController, MainController, AdminController with FXML views), Business Logic Layer (Facade and BizLogic classes handling rules and validation), Data Access Layer (DatabaseConnector and DatabaseUtils managing database operations), and Data Layer (Azure MySQL Database). This separation ensures maintainability and clear responsibility boundaries.

2.2 Design Patterns

Facade Pattern: The Facade class provides a unified interface to business logic, routing actions ("addFlight", "removeFlight", "searchFlight", etc.) to appropriate methods while centralizing exception handling.

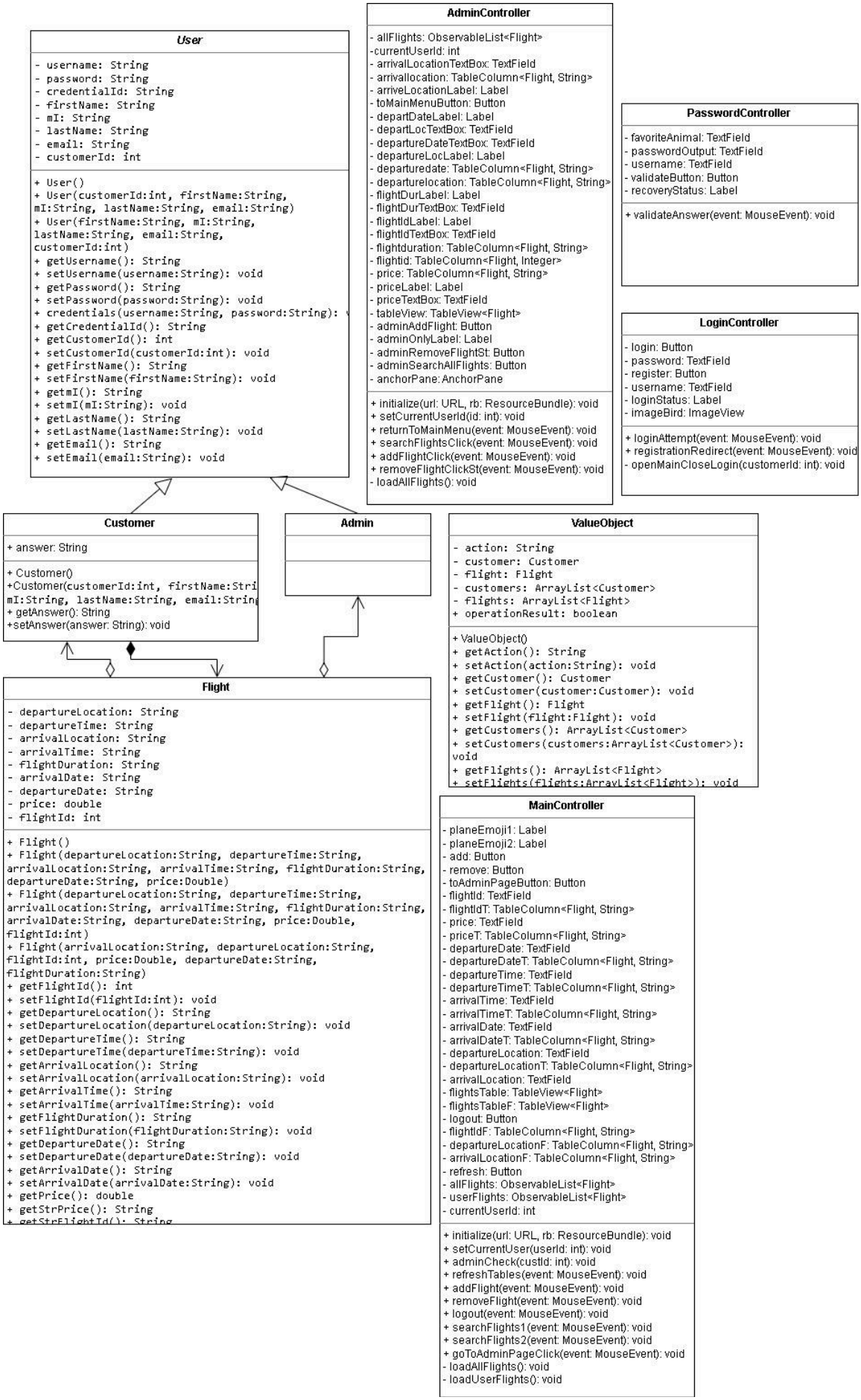
MVC Pattern: Models (Customer, Admin, Flight, ValueObject) represent data structures, Views (FXML files) define layouts, and Controllers handle interactions and coordinate between model and view.

Observer Pattern: JavaFX's ObservableList automatically updates TableViews when data changes, eliminating manual refresh operations.

Inheritance: Admin extends Customer, allowing administrators to function as specialized customers with additional privileges while code treats them polymorphically.

---

3. Class UML Diagram


User
- username: String
- password: String
- credentialId: String
- firstName: String
- mI: String
- lastName: String
- email: String
- customerId: int
+ User()
+ User(customerId:int, firstName:String, mI:String, lastName:String, email:String)
+ User(firstName:String, mI:String, lastName:String, email:String, customerId:int)
+ getUsername(): String
+ setUsername(username:String): void
+ getPassword(): String
+ setPassword(password:String): void
+ credentials(username:String, password:String):
+ getCredentialId(): String
+ getCustomerId(): int
+ setCustomerId(customerId:int): void
+ getFirstName(): String
+ setFirstName(firstName:String): void
+ getmI(): String
+ setmI(mI:String): void
+ getLastName(): String
+ setLastName(lastName:String): void
+ getEmail(): String
+ setEmail(email:String): void
AdminController
- allFlights: ObservableList<Flight>
-currentUserId: int
- arrivalLocationTextBox: TextField
- arrivallocation: TableColumn<Flight, String>
- arriveLocationLabel: Label
- toMainMenuButton: Button
- departDateLabel: Label
- departLocTextBox: TextField
- departureDateTextBox: TextField
- departureLocLabel: Label
- departuredate: TableColumn<Flight, String>
- departurelocation: TableColumn<Flight, String>
- flightDurLabel: Label
- flightDurTextBox: TextField
- flightIdLabel: Label
- flightIdTextBox: TextField
- flightduration: TableColumn<Flight, String>
- flightid: TableColumn<Flight, Integer>
- price: TableColumn<Flight, String>
- priceLabel: Label
- priceTextBox: TextField
- tableView: TableView<Flight>
- adminAddFlight: Button
- adminOnlyLabel: Label
- adminRemoveFlightSt: Button
- adminSearchAllFlights: Button
- anchorPane: AnchorPane
+ initialize(url: URL, rb: ResourceBundle): void
+ setCurrentUserId(id: int): void
+ returnToMainMenu(event: MouseEvent): void
+ searchFlightsClick(event: MouseEvent): void
+ addFlightClick(event: MouseEvent): void
+ removeFlightClickSt(event: MouseEvent): void
- loadAllFlights(): void
PasswordController
- favoriteAnimal: TextField
- passwordOutput: TextField
- username: TextField
- validateButton: Button
- recoveryStatus: Label
+ validateAnswer(event: MouseEvent): void
LoginController
- login: Button
- password: TextField
- register: Button
- username: TextField
- loginStatus: Label
- imageBird: ImageView
+ loginAttempt(event: MouseEvent): void
+ registrationRedirect(event: MouseEvent): void
- openMainCloseLogin(customerId: int): void
Customer
+ answer: String
+ Customer()
+Customer(customerId:int, firstName:Stri
mI:String, lastName:String, email:String
+ getAnswer(): String
+setAnswer(answer: String): void
Admin
ValueObject
- action: String
- customer: Customer
- flight: Flight
- customers: ArrayList<Customer>
- flights: ArrayList<Flight>
+ operationResult: boolean
+ ValueObject()
+ getAction(): String
+ setAction(action:String): void
+ getCustomer(): Customer
+ setCustomer(customer:Customer): void
+ getFlight(): Flight
+ setFlight(flight:Flight): void
+ getCustomers(): ArrayList<Customer>
+ setCustomers(customers:ArrayList<Customer>): void
+ getFlights(): ArrayList<Flight>
+ setFlights(flights:ArrayList<Flight>): void
Flight
- departureLocation: String
- departureTime: String
- arrivalLocation: String
- arrivalTime: String
- flightDuration: String
- arrivalDate: String
- departureDate: String
- price: double
- flightId: int
+ Flight()
+ Flight(departureLocation:String, departureTime:String, arrivalLocation:String, arrivalTime:String, flightDuration:String, departureDate:String, price:Double)
+ Flight(departureLocation:String, departureTime:String, arrivalLocation:String, arrivalTime:String, flightDuration:String, arrivalDate:String, departureDate:String, price:Double, flightId:int)
+ Flight(arrivalLocation:String, departureLocation:String, flightId:int, price:Double, departureDate:String, flightDuration:String)
+ getFlightId(): int
+ setFlightId(flightId:int): void
+ getDepartureLocation(): String
+ setDepartureLocation(departureLocation:String): void
+ getDepartureTime(): String
+ setDepartureTime(departureTime:String): void
+ getArrivalLocation(): String
+ setArrivalLocation(arrivalLocation:String): void
+ getArrivalTime(): String
+ setArrivalTime(arrivalTime:String): void
+ getFlightDuration(): String
+ setFlightDuration(flightDuration:String): void
+ getDepartureDate(): String
+ setDepartureDate(departureDate:String): void
+ getArrivalDate(): String
+ setArrivalDate(arrivalDate:String): void
+ getPrice(): double
+ getStrPrice(): String
+ getStrFlightId(): String
MainController
- planeEmoji1: Label
- planeEmoji2: Label
- add: Button
- remove: Button
- toAdminPageButton: Button
- flightId: TextField
- flightIdT: TableColumn<Flight, String>
- price: TextField
- priceT: TableColumn<Flight, String>
- departureDate: TextField
- departureDateT: TableColumn<Flight, String>
- departureTime: TextField
- departureTimeT: TableColumn<Flight, String>
- arrivalTime: TextField
- arrivalTimeT: TableColumn<Flight, String>
- arrivalDate: TextField
- arrivalDateT: TableColumn<Flight, String>
- departureLocation: TextField
- departureLocationT: TableColumn<Flight, String>
- arrivalLocation: TextField
- flightsTable: TableView<Flight>
- flightsTableF: TableView<Flight>
- logout: Button
- flightIdF: TableColumn<Flight, String>
- departureLocationF: TableColumn<Flight, String>
- arrivalLocationF: TableColumn<Flight, String>
- refresh: Button
- allFlights: ObservableList<Flight>
- userFlights: ObservableList<Flight>
- currentUserId: int
+ initialize(url: URL, rb: ResourceBundle): void
+ setCurrentUser(userId: int): void
+ adminCheck(custId: int): void
+ refreshTables(event: MouseEvent): void
+ addFlight(event: MouseEvent): void
+ removeFlight(event: MouseEvent): void
+ logout(event: MouseEvent): void
+ searchFlights1(event: MouseEvent): void
+ searchFlights2(event: MouseEvent): void
+ goToAdminPageClick(event: MouseEvent): void
- loadAllFlights(): void
- loadUserFlights(): void

4. Database Model

4.1 Customers Table

Purpose: Store customer personal information

Column Name	Data Type	Constraints	Description
customer_id	INT	PRIMARY KEY	Unique customer identifier
first_name	VARCHAR(100)		Customer's first name
middle_initial	VARCHAR(1)		Customer's middle initial
last_name	VARCHAR(100)		Customer's last name
email	VARCHAR(100)		Customer's email
admin	TINYINT(1)		Boolean denoting admin access

4.2 Flights Table

Purpose: Store all available flight information

Column Name	Data Type	Constraints	Description
flightid	INT	PRIMARY KEY	Unique flight identifier
departurelocation	VARCHAR(100)		Origin airport
departuretime	VARCHAR(100)		Departure time
arrivallocation	VARCHAR(100)		Destination airport
arrivaltime	VARCHAR(26)		Arrival time
flightduration	VARCHAR(100)		Total flight duration

departuredate	VARCHAR(26)		Date of departure
price	VARCHAR(65)		Flight ticket price

4.3 Credentials Table

Purpose: Store login credentials for customers

Column Name	Data Type	Constraints	Description
credentials_id	INT	PRIMARY KEY	Unique credential identifier
customer_id	INT	FOREIGN KEY	References customer
username	VARCHAR(100)		User's login username
password	VARCHAR(100)		User's login password
security_answer	VARCHAR(20)		User's security answer question

4.4 Flight_customer Table

Purpose: Store login credentials for customers

Column Name	Data Type	Constraints	Description
my_row_id	BIGINT	PRIMARY KEY	Unique booking record ID
flightid	INT	FOREIGN KEY	References flights
customer_id	INT	FOREIGN KEY	References customers
capacity	INT		Current booking capacity
is_max_capacity	TINYINT(1)		Whether flight is at capacity

5. Program Functionality and Flow

5.1 Startup and Login

Application displays splash screen during initialization, then presents main menu with registration and login options. Registration collects comprehensive user data including security questions for password recovery. Login validates username/password against database through Facade → BizLogic → DatabaseUtils chain. Successful login opens main page with setCurrentUser(customerId), loading user's booked flights. Password recovery uses security question validation to restore access.

5.2 Main Page and Flight Display

MainController.initialize() configures TableColumn cell value factories, creates ObservableLists (allFlights, userFlights), and calls loadAllFlights(). This method queries "SELECT * FROM flights", creates Flight objects from ResultSet, and populates flightsTable. When user logs in, setCurrentUser() calls loadUserFlights() which joins flights and flight_customer tables filtered by customer_id, populating flightsTableF with user's bookings.

5.3 Search Functionality

Two search interfaces filter flights dynamically. searchFlights1() collects criteria from text fields, calls DatabaseUtils.searchAllFlights() which builds dynamic SQL query ("WHERE 1=1" with appended conditions for non-empty fields), returns ArrayList<Flight>, and displays results in flightsTable. searchFlights2() similarly filters user's bookings using searchCustomerFlights() which adds customer_id filter to joined query, displaying in flightsTableF.

5.4 Booking Operations

addFlight(): Gets selected flight, creates ValueObject with action="addFlight", passes to Facade. BizLogic.addFlightToCustomer() validates: (1) customerHasFlight() checks duplicates, (2) flightHasCapacity() checks capacity, (3) checks scheduling conflicts comparing dates/times with existing bookings. If valid, addFlightToCustomer() inserts into flight_customer table. Success adds flight to userFlights ObservableList, auto-updating UI.

removeFlight(): Gets selected flight from flightsTableF, creates ValueObject with action="removeFlight", passes to Facade. BizLogic.removeFlightFromCustomer() calls DatabaseUtils.removeFlightFromCustomer() which deletes from flight_customer table. Success removes from userFlights ObservableList, auto-updating UI.

5.5 Admin Operations

goToAdminPageClick() loads admin-page.fxml, passes currentUserId to AdminController. Admin page displays all flights with search, add, and remove functionality. Administrators can add flights by entering details, remove flights (cascading to bookings), and search flight inventory. returnToMainMenu() navigates back to main page, preserving user session.

5.6 Refresh and Logout

refreshTables() calls loadAllFlights() and loadUserFlights() to reload data from database, refreshing both tables. logout() creates new LoginApplication, starts it on new Stage, closes current main page Stage, returning user to login screen.

---

6. Exception Handling

6.1 Exception Strategy

Custom exceptions represent specific error conditions: LoginException, RegistrationException, AddFlightRecordDuplicationException (duplicate bookings), AddToFlightListException (capacity issues), RemoveFlightRecordException, FlightSchedulingException (time conflicts), and customer record exceptions. Facade catches all exceptions from BizLogic, sets vo.operationResult=false, logs error messages. UI checks operationResult and displays appropriate feedback.

6.2 Best Practices

All database operations use try-with-resources for automatic resource cleanup. PreparedStatements prevent SQL injection. Specific exceptions enable targeted error messages. Facade centralizes exception handling, reducing UI complexity. Technical details logged to console while user-friendly messages display in interface.

---

7. Conclusion

7.1 Achievements

CheepSkies successfully delivers a complete flight booking solution with secure authentication, comprehensive search capabilities, robust booking validation (duplicate prevention, capacity tracking, conflict detection), role-based access control, and persistent data storage. The system

implements established design patterns (Facade, MVC, Observer, Inheritance) promoting maintainability and extensibility.

7.2 Technical Strengths

Layered architecture with clear separation of concerns, comprehensive exception handling, security through PreparedStatements and credential validation, responsive UI with automatic updates via ObservableList, flexible search with dynamic query building, and proper transaction management maintaining data integrity.

7.3 Future Enhancements

Potential improvements include payment integration, email notifications for bookings, advanced reporting and analytics, mobile responsive design, multi-language support, and enhanced admin dashboards for monitoring system usage and managing customer accounts.

Document Version: 1.0
Last Updated: December 2025
Project: CheepSkies Flight Booking System